Dataset Import

```
# !pip install kagglehub --upgrade -q
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("undefinenull/million-song-dataset-spoti

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/undefinenull/million-song-dataset-spotify-lastfm/1.archive...
100%|██████████| 639M/639M [00:07<00:00, 94.7MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/undefinenull/million-song-dataset-spotify-lastfm/versions/
```

```
import os
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# data_path = Path('kagglehub/datasets/undefinenull/million-song-dataset-s

# songs_data_path = data_path / 'Music Info.csv'
# users_data_path = data_path / 'User Listening History.csv'
```

Songs dataset

```
from pathlib import Path

songs_data_path = Path(path) / 'Music Info.csv'

df_song = pd.read_csv(songs_data_path)
df_song.head()
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre	y
0	TRIOREW128F424EAF0	Mr. Brightside	The Killers	https://p.scdn.co/mp3-preview/4d26180e6961fd46...	09ZQ5TmUG8TSL56n0knqrj	rock, alternative, indie, alternative_rock, in...	NaN	2
1	TRRIVDJ128F429B0E8	Wonderwall	Oasis	https://p.scdn.co/mp3-preview/d012e536916c927b...	06UfBBDISthj1ZJAtX4xjj	rock, alternative, indie, pop, alternative_roc...	NaN	2
2	TROUVHL128F426C441	Come as You Are	Nirvana	https://p.scdn.co/mp3-preview/a1c11bb1cb231031...	0keNu0t0tqsWtExGM3nT1D	rock, alternative, alternative_rock, 90s, grunge	RnB	1
3	TRUEIND128F93038C4	Take Me Out	Franz Ferdinand	https://p.scdn.co/mp3-preview/399c401370438be4...	0ancVQ9wEcHVd0RrGICTE4	rock, alternative, indie, alternative_rock, in...	NaN	2
4	TRLNZBD128F935E4D8	Creep	Radiohead	https://p.scdn.co/mp3-preview/e7eb60e9466bc3a2...	01QoK9DA7VTeTSE3MNzp4I	rock, alternative, indie, alternative_rock, in...	RnB	2

5 rows × 21 columns

```
df_song.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50683 entries, 0 to 50682
Data columns (total 21 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   track_id             50683 non-null  object 
 1   name                 50683 non-null  object 
 2   artist               50683 non-null  object 
 3   spotify_preview_url  50683 non-null  object 
 4   spotify_id           50683 non-null  object 
 5   tags                 49556 non-null  object 
 6   genre                22348 non-null  object 
 7   year                 50683 non-null  int64  
 8   duration_ms          50683 non-null  int64  
 9   danceability         50683 non-null  float64
 10  energy               50683 non-null  float64
 11  key                  50683 non-null  int64  
 12  loudness             50683 non-null  float64
 13  mode                 50683 non-null  int64  
 14  speechiness          50683 non-null  float64
 15  acousticness         50683 non-null  float64
 16  instrumentalness     50683 non-null  float64
 17  liveness             50683 non-null  float64
 18  valence              50683 non-null  float64
 19  tempo                50683 non-null  float64
 20  time_signature       50683 non-null  int64  
dtypes: float64(9), int64(5), object(7)
memory usage: 8.1+ MB
```

```python
column_to_drop = 'spotify_preview_url'
if column_to_drop in df_song.columns:
    df_song = df_song.drop(columns=[column_to_drop])

print("shape: ", df_song.shape)
print()

# print("info: ", df_song.info())
# print()

print("describe: ", df_song.describe())
print()

print("null: ", df_song.isna().sum())
print()

import missingno as msno
msno.matrix(df_song)
```

```
shape:  (50683, 20)

describe:                year   duration_ms  danceability        energy           key  \
count  50683.000000  5.068300e+04  50683.000000  50683.000000  50683.000000
mean    2004.017323  2.511551e+05      0.493537      0.686486      5.312748
std        8.860172  1.075860e+05      0.178838      0.251808      3.568078
min     1900.000000  1.439000e+03      0.000000      0.000000      0.000000
25%     2001.000000  1.927330e+05      0.364000      0.514000      2.000000
50%     2006.000000  2.349330e+05      0.497000      0.744000      5.000000
75%     2009.000000  2.881930e+05      0.621000      0.905000      9.000000
max     2022.000000  3.816373e+06      0.986000      1.000000     11.000000

           loudness          mode   speechiness  acousticness  \
count  50683.000000  50683.000000  50683.000000  50683.000000
mean      -8.291204      0.631060      0.076023      0.213808
std        4.548365      0.482522      0.076007      0.302848
min      -60.000000      0.000000      0.000000      0.000000
25%      -10.375000      0.000000      0.035200      0.001400
50%       -7.200000      1.000000      0.048200      0.039900
75%       -5.089000      1.000000      0.083500      0.340000
max        3.642000      1.000000      0.954000      0.996000

       instrumentalness      liveness       valence         tempo  \
count      50683.000000  50683.000000  50683.000000  50683.000000
mean           0.225283      0.215425      0.433134    123.507682
```

```
# ratio of missing values

(
    df_song
    .isna()
    .mean()
    .sort_values(ascending=False)
    .head(2)
    .mul(100)
)
```

```
max          5.000000
```

0

genre	55.906320
artist	2.223625

dtype: float64

```
# check for duplicates based on name of song

(
    df_song
    .assign(name=df_song['name'].str.lower())
    .duplicated(subset='name')
    .sum()
)
```

```
0
```

```
# fetching that rows that are duplicate

(
    df_song
    .loc[df_song
         .assign(name=df_song['name'].str.lower())
         .duplicated(subset='name',keep=False)
    ]
    .assign(name=df_song['name'].str.lower())
    .sort_values(by='name')
)
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms	d
34480	TRKKZMK128F4257579	3 am	Liv Kristine	1TgsnkiolcBhQssCR37JXx	female_vocalists, power_metal, gothic_metal, g...	NaN	2005	302880	
6588	TRGGALK12903CB68E5	3 am	Matchbox Twenty	5vYA1mW9g2Coh1HUFUSmlb	rock, alternative, pop, alternative_rock, 90s,...	NaN	1996	225946	
29795	TRLOXMF128F934BF04	3am	Matchbox Twenty	5vYA1mW9g2Coh1HUFUSmlb	rock, alternative, 90s, piano, american, pop_rock	NaN	1996	225946	
43800	TRPWOAS128E0781045	3am	Halsey	1OfLNb6dQ9dra1B58iT9Ex	pop_rock	NaN	2020	234858	
15088	TRJRBRX128F4255789	4th of july	Soundgarden	237oH9rNUYpIBeHfAn3WJ0	hard_rock, 90s, grunge	NaN	1994	308866	
...	...	...	...	...	...	...	...	...	...
34772	TRJFGYO128F4259743	you're the one that i want	Lo-Fang	1dEHQktvcM8vCCyI1x7yVB	indie, male_vocalists, cover	NaN	2014	204706	
32677	TRFZYLL128F146902A	you've really got a hold on me	Smokey Robinson and The Miracles	03AkIZeRvGpTvDF9vNJtdj	soul, 60s, oldies	Rock	2012	179513	
20976	TRUXHGS128F145E41A	you've really got a hold on me	The Miracles	01FtGX94CSvO5Zxs5B6AMM	soul, 60s, oldies	NaN	1994	180266	
27680	TRXWPMW12903CB42A0	zombies	Childish Gambino	73kAUSAht4YOR7xNPmNb2L	funk	NaN	2016	281813	
37428	TRORIUQ128F42620A3	zombies	Lacuna Coil	02J446tHHPvSSNeLxW8Lg6	gothic_metal, gothic	NaN	2014	227440	

1614 rows × 20 columns

```
# some title are same but diff. by spacing so diff. way to find duplicates
(
    df_song
    .duplicated(subset='spotify_id')
    .sum()
)
```

```
np.int64(9)
```

```
(
    df_song
    .duplicated(subset=['spotify_id','year','duration_ms'])
    .sum()
)
```

```
np.int64(9)
```

```
# so 9 rows are duplicate
(
    df_song
    .loc[df_song
    .duplicated(subset=['spotify_id','year','duration_ms'],keep=False)]
    .sort_values(by=['spotify_id','year','duration_ms'])
)
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms	da
15326	TRJNHPN128F92EF139	Adagio For Strings	Samuel Barber	00otCiz9kUb3Vg7LPKNCZG	instrumental, classical, soundtrack, beautiful	NaN	2014	431412	
21570	TRLSRQD128F426CFF8	Adagio for Strings, Op. 11	Samuel Barber	00otCiz9kUb3Vg7LPKNCZG	classical	NaN	2014	431412	
14861	TRLOZQZ128F92E8A3F	How Do You Want It	2Pac	02VsIBmSkhc7uHNyPViZR3	rap, hip_hop	NaN	2011	289000	
14981	TRXHJQY128F42B5094	How Do U Want It	2Pac	02VsIBmSkhc7uHNyPViZR3	rap, hip_hop, american	Rap	2011	289000	
37040	TRGCZFO128F92EE221	Je pense à toi	Amadou & Mariam	09jsAIZF9ThihIzdrw4KAS	alternative, beautiful, french	NaN	2005	316880	
49162	TRZBNQU128F148C04F	Je Pense A Toi	Amadou & Mariam	09jsAIZF9ThihIzdrw4KAS	NaN	NaN	2005	316880	
13427	TRJQFIT128E0781CED	Too Much Too Young	The Specials	0ndKJL8gA4zLl317M7vndn	punk, 80s, new_wave, reggae, ska	NaN	2012	116160	
46512	TRDTUTO128F422F138	Too Much Too Young (Live)	The Specials	0ndKJL8gA4zLl317M7vndn	ska	NaN	2012	116160	
1684	TRRZUGN128F42A1EEE	There There	Radiohead	0thdzbW0cRKCx12VbBRB6T	rock, electronic, alternative, indie, alternat...	Rock	2008	323600	
2983	TRXFHCL128F92E0989	There, There	Radiohead	0thdzbW0cRKCx12VbBRB6T	rock, electronic, alternative, indie, alternat...	NaN	2008	323600	
23200	TRXUYQW128F42370DB	hHallmark	Broken Social Scene	1Ntzk4JoxcAsrWi73MoBjr	alternative, indie, ambient, soundtrack, psych...	Rock	2004	233706	
26389	TRCUHWL128F4249F1A	Hallmark	Broken Social Scene	1Ntzk4JoxcAsrWi73MoBjr	indie, alternative_rock, instrumental, post_rock	NaN	2004	233706	
4300	TRHNAWF128F423EAE4	Do You Love Me Now?	The Breeders	22Ty5gK6zbw0hRtGypTuX5	rock, alternative, indie, female_vocalists, al...	Rock	1993	181800	
42187	TRVENST12903D13BBB	Do You Love Me Now	The Breeders	22Ty5gK6zbw0hRtGypTuX5	rock, alternative, indie, female_vocalists, al...	NaN	1993	181800	
29710	TRRLAHF12903CAFEA1	Greatest Hit	Annie	3MUviQJP5DSYI3Li4EbYTQ	electronic, pop, dance, house	NaN	2004	220293	
32191	TRCEFVZ128F4283203	The Greatest Hit	Annie	3MUviQJP5DSYI3Li4EbYTQ	electronic, pop, female_vocalists, dance, electro	NaN	2004	220293	
6588	TRGGALK12903CB68E5	3 AM	Matchbox Twenty	5vYA1mW9g2Coh1HUFUSmlb	rock, alternative, pop, alternative_rock, 90s,...	NaN	1996	225946	
29795	TRLOXMF128F934BF04	3AM	Matchbox Twenty	5vYA1mW9g2Coh1HUFUSmlb	rock, alternative, 90s, piano, american, pop_rock	NaN	1996	225946	

```
df_song.drop_duplicates(subset=['spotify_id','year','duration_ms'],inplace=
```

```
(
    df_song
    .duplicated(subset='spotify_id')
    .sum()
)
```

```
np.int64(0)
```

```
df_song.columns
```

```
Index(['track_id', 'name', 'artist', 'spotify_id', 'tags', 'genre', 'year',
       'duration_ms', 'danceability', 'energy', 'key', 'loudness', 'mode',
       'speechiness', 'acousticness', 'instrumentalness', 'liveness',
       'valence', 'tempo', 'time_signature'],
      dtype='object')
```

```
df_song.dtypes
```

	0
track_id	object
name	object
artist	object
spotify_id	object
tags	object
genre	object
year	int64
duration_ms	int64
danceability	float64
energy	float64
key	int64
loudness	float64
mode	int64
speechiness	float64
acousticness	float64
instrumentalness	float64
liveness	float64
valence	float64
tempo	float64
time_signature	int64

dtype: object

```
categorical_columns = df_song.select_dtypes(include='object').columns
```

```
def categorical_analysis(df,c_name,k=15):
  for i in c_name:
    print("*"*20)
    print(f"no. of categories in column {i} are",df[i].str.lower().nunique
    if i in ['artist','genre']:
      print(df[i].value_counts().head(k))
    if i == 'genre':
      print(f"the unique categories in {i} are {df[i].dropna().unique()}")
      print()

```

```
print(f'shape: {df_song.shape}')
categorical_analysis(df_song,categorical_columns)
```

```
shape: (50674, 20)
********************
no. of categories in column track_id are 50674
********************
no. of categories in column name are 49860
********************
no. of categories in column artist are 8317
artist
The Rolling Stones    132
Radiohead             110
```

```
Autechre              105
Tom Waits             100
Bob Dylan              98
The Cure               94
Metallica              85
Johnny Cash            84
Nine Inch Nails        83
Sonic Youth            81
Iron Maiden            76
In Flames              76
Elliott Smith          76
Mogwai                 75
Boards of Canada       75
Name: count, dtype: int64
********************
no. of categories in column spotify_id are 50674
********************
no. of categories in column tags are 20054
********************
no. of categories in column genre are 15
genre
Rock          9965
Electronic    3710
Metal         2516
Pop           1145
Rap            820
Jazz           793
RnB            696
Reggae         691
Country        607
Punk           383
Folk           355
New Age        237
Blues          189
World          140
Latin          100
Name: count, dtype: int64
the unique categories in genre are ['RnB' 'Rock' 'Pop' 'Metal' 'Electronic' 'Jazz' 'Punk' 'Country' 'Folk'
 'Reggae' 'Rap' 'Blues' 'New Age' 'Latin' 'World']
```

```
# we can't track similarity in unique column, so no need of that.
```

```
df_song.groupby('genre')[['genre','tags']].sample(3)
# here there is no relation of genre with tags
```

	genre	tags
5849	Blues	rock, alternative, indie, alternative_rock, in...
23796	Blues	classic_rock, blues, guitar, blues_rock
25872	Blues	alternative, punk, japanese
39289	Country	electronic, idm
33222	Country	country
33218	Country	country
41657	Electronic	chillout, lounge
29248	Electronic	electronic, trance
39146	Electronic	electronic, trip_hop, downtempo, idm, lounge
45728	Folk	new_age
32735	Folk	rock, classic_rock, 60s, oldies
45489	Folk	folk, new_age
9205	Jazz	jazz
31467	Jazz	electronic, techno
18062	Jazz	progressive_rock
49166	Latin	NaN
45953	Latin	rock, ska
36735	Latin	pop, 80s, french
19479	Metal	metal, death_metal, heavy_metal, thrash_metal,...
26794	Metal	progressive_metal, gothic_metal, symphonic_metal
25120	Metal	metalcore, screamo
45684	New Age	ambient, new_age
26356	New Age	indie, instrumental, post_rock
22028	New Age	classical, new_age
25918	Pop	electronic, japanese, j_pop
41069	Pop	pop, swedish
24171	Pop	heavy_metal, thrash_metal, power_metal
47251	Punk	rock, alternative, indie, indie_rock
9093	Punk	jazz, ambient, chillout, trip_hop, downtempo, ...
23492	Punk	rock, punk, punk_rock, reggae, post_punk
10961	Rap	electronic, ambient, chillout, trip_hop, downt...
14433	Rap	rap, 90s, hip_hop
18168	Rap	instrumental, progressive_rock, progressive_metal
45967	Reggae	ska
30809	Reggae	reggae
46389	Reggae	ska
20110	RnB	rock, alternative, alternative_rock, punk, ind...
37487	RnB	rock, gothic_metal
9160	RnB	female_vocalists, jazz, soul, chillout, funk, ...
33553	Rock	rock, indie, indie_rock, country, britpop
30954	Rock	alternative_rock, indie_rock, american
21293	Rock	electronic, chillout, trip_hop
39842	World	electronic, dark_ambient
49120	World	NaN
29319	World	trance

```
1 # non english title, doesn't matter because we need artist for similarity
2
3 (
```

```
    df_song
    .loc[df_song['name'].str.contains('[^\d\s\w.!@#$%^&*(){},.-_=\\|/?<>;:\'"]+')]

)
```

```
<>:5: SyntaxWarning: invalid escape sequence '\d'
<>:5: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_719/1806308408.py:5: SyntaxWarning: invalid escape sequence '\d'
  .loc[df_song['name'].str.contains('[^\d\s\w.!@#$%^&*(){},.-_=\\|/?<>;:\'"]+')]
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms
47	TRADPIA128E078EE1B	Heart-Shaped Box	Nirvana	0FMu3Z1yvTJKLWgxG6ZLS5	rock, alternative, alternative_rock, 90s, grunge	Rock	2007	278520
536	TRJFMNK128E07840C2	19-2000	Gorillaz	1kSWrMDt439cT64WpjJigW	rock, electronic, alternative, indie, pop, alt...	Rock	2001	210880
734	TRILLQO12903CA55EA	Ob-La-Di, Ob-La-Da	The Beatles	7xZYkwsvddFcgQouUiWrmI	rock, pop, classic_rock, british, 60s, oldies,...	Pop	2014	185844
938	TRRISKY128EF3434F1	E-Pro	Beck	01MBhRpvFkbeRwAp7gcF2W	rock, electronic, alternative, indie, alternat...	Rock	2005	202360
1231	TRCVTGV128F4279C82	I-E-A-I-A-I-O	System of a Down	17cKTJlWIHTeTmdgNs77DL	rock, alternative, metal, alternative_rock, ha...	NaN	2002	188600
...	...	...	...	...	...	...	...	...
49631	TRMSPDS128F932C84E	Sacrapos - The Disparaging Last Gaze	Eluveitie	1NJTCZVrr0Pa4WUey4wapt	folk	Rock	2009	163306
50025	TRSWDOD128F932F295	Pitter-Pat	Erin McCarley	0y1HE8lUCUGUcOgqkFn4yC	female_vocalists, singer_songwriter	Pop	2009	259800
50598	TRWKLWW128F42754DF	Mission De-Evolution	Knights Of The Abyss	3HsSq883okxAfYvREq7Akd	metal, death_metal	NaN	2007	291306
50625	TRJSXNV128F932B41E	CRAZY-FLAG	ギルガメッシュ	11JuKOEz2IqadIbCNWkSel	NaN	NaN	2014	219291
50658	TRKSMNP128F4298B11	Heart-Shaped Gear	9mm Parabellum Bullet	1JhQ7CWI9FXVqWUQEj2wrP	rock, japanese	NaN	2007	227653

446 rows × 20 columns

```
(    df_song
    .loc[df_song['artist'].str.contains('[^\d\s\w.!@#$%^&*(){},.-_=\\|/?<>
    .groupby('artist')
    .size()
    .sort_values(ascending=False).head(20)
)
# doesn't matter because we will encode it
```

```
<>:2: SyntaxWarning: invalid escape sequence '\d'
<>:2: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_719/3556878058.py:2: SyntaxWarning: invalid escape sequence '\d'
  .loc[df_song['artist'].str.contains('[^\d\s\w.!@#$%^&*(){},.-_=\\|/?<>;:"]+')]
```

artist	0
blink-182	67
Guns N' Roses	51
De-Phazz	39
Ska-P	35
At the Drive-In	32
Jay-Z	30
Melt-Banana	30
The All-American Rejects	27
L'Arc~en~Ciel	22
Florence + the Machine	21
Blackmore's Night	19
Plain White T's	18
L'Âme Immortelle	17
Static-X	17
Martina Topley-Bird	15
Anti-Flag	15
µ-Ziq	14
Jane's Addiction	13
Destiny's Child	13
Jack's Mannequin	13

dtype: int64

```
all_tags = []

for tags in df_song['tags'].dropna().str.replace(" ","").str.split(","):
  all_tags.extend(tags)

print('The no. of unique tags are ',len(set(all_tags)))
```

```
The no. of unique tags are  100
```

```
numerical_columns = df_song.select_dtypes(exclude='object').columns
numerical_columns
```

```
Index(['year', 'duration_ms', 'danceability', 'energy', 'key', 'loudness',
       'mode', 'speechiness', 'acousticness', 'instrumentalness', 'liveness',
       'valence', 'tempo', 'time_signature'],
      dtype='object')
```

```
int_col = df_song.select_dtypes(include='int').columns
int_col
```

```
Index(['year', 'duration_ms', 'key', 'mode', 'time_signature'], dtype='object')
```

```
df_song[int_col].head()
```

	year	duration_ms	key	mode	time_signature
0	2004	222200	1	1	4
1	2006	258613	2	1	4
2	1991	218920	4	0	4
3	2004	237026	9	1	4
4	2008	238640	7	1	4

key :
if low -> sa, re, ga ...
if high -> ni, sa ...

mode :
if 0 -> tension or sad song
if 1 -> happy or cheerful song

time signature :
4 -> regular bits
low -> low bits
high -> complex bits

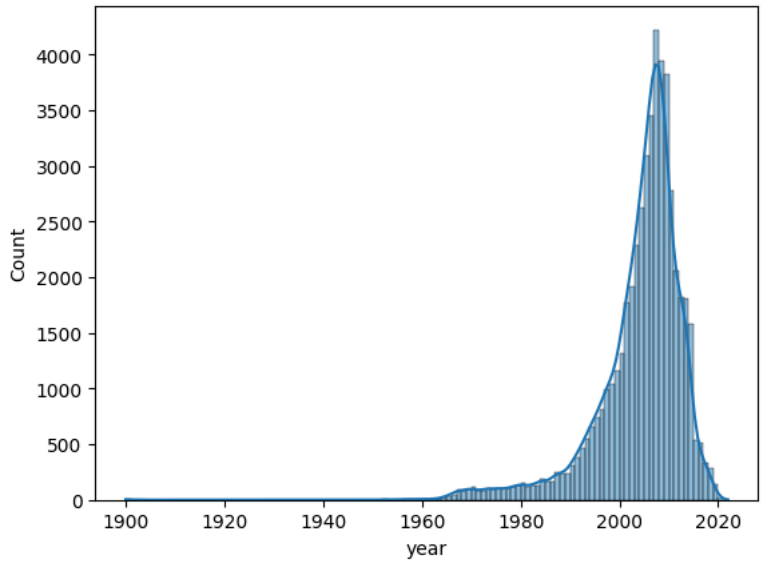
```
# stastical summary:

(
    df_song
    .loc[:,int_col]
    .drop(columns='duration_ms')
    .assign(**{
        col:df_song[col].astype('object') for col in int_col.drop('duratio
    })
    .describe()
)
```

	year	key	mode	time_signature
count	50674	50674	50674	50674
unique	75	12	2	5
top	2007	9	1	4
freq	4221	5907	31979	44981

```
# range of data
(
    df_song
    .loc[:,int_col]
    .assign(duration_minutes=df_song['duration_ms'].div(1000).div(60))
    .drop(columns='duration_ms')
    .agg(['min','max'])
)
```

	year	key	mode	time_signature	duration_minutes
min	1900	0	0	0	0.023983
max	2022	11	1	5	63.606217

```
# no. of songs per year
sns.histplot(df_song['year'],kde=True,stat='count',bins=df_song['year'].ma
plt.show()
```




```
(df_song.loc[:,'year'].value_counts().head(5))
```

year	count
2007	4221
2008	3947
2009	3827
2006	3453
2005	3085

dtype: int64

key values:

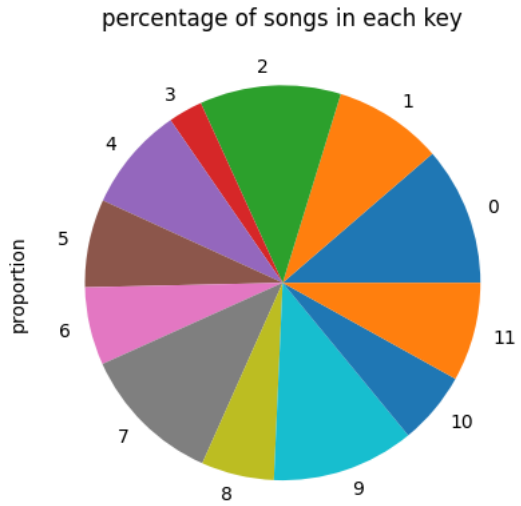
Key Value	English Note	Hindi Note (Sargam)
0	C	सा (Sa)
1	C# / Db	कोमल रे (komal Re)
2	D	रे (Re)
3	D# / Eb	कोमल ग (komal Ga)
4	E	ग (Ga)
5	F	म (Ma)
6	F# / Gb	तीव्र म (tivra Ma)
7	G	प (Pa)
8	G# / Ab	कोमल ध (komal Dha)
9	A	ध (Dha)
10	A# / Bb	कोमल नी (komal Ni)
11	B	नी (Ni)

```
(
    np.sort(df_song['key'].unique())
)
```

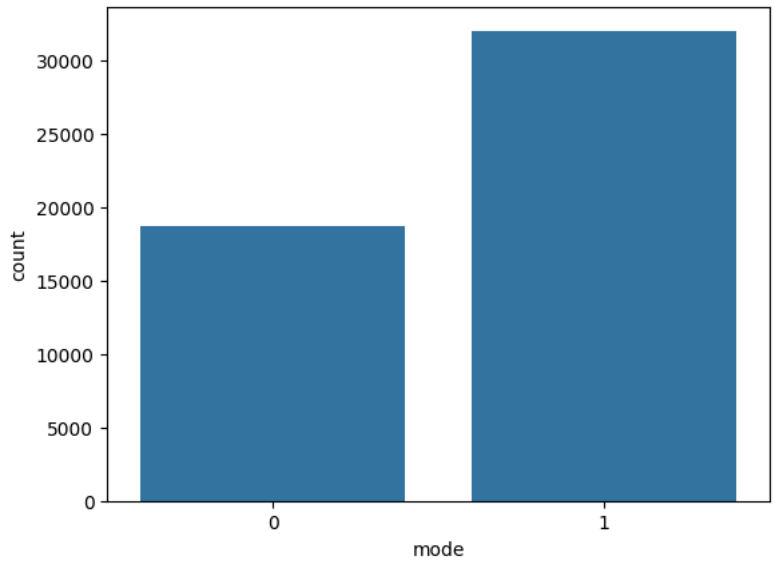
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
(
    df_song['key']
    .value_counts(normalize=True)
    .mul(100)
    .sort_index()
    .plot(kind='pie',title='percentage of songs in each key')
 )

```

```
<Axes: title={'center': 'percentage of songs in each key'}, ylabel='proportion'>
```




```
1 sns.countplot(df_song,x='mode')
2 plt.show()
```



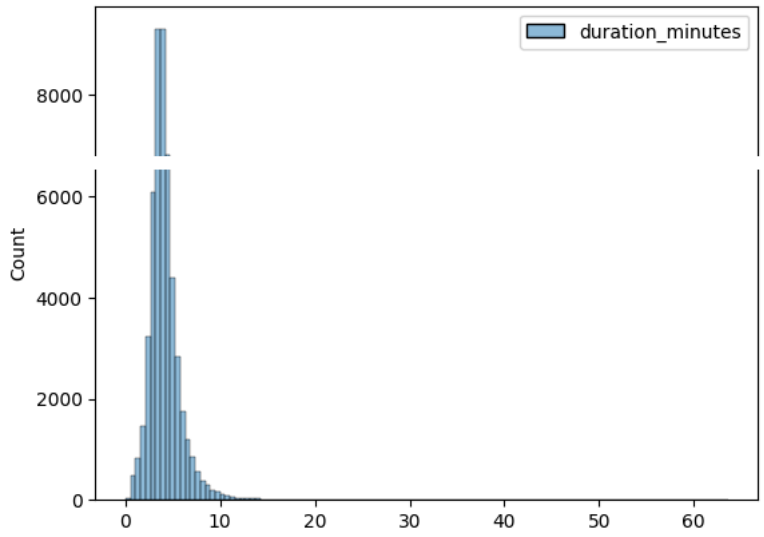

```
1 (
2     df_song['time_signature']
3     .value_counts(normalize=True)
4     .mul(100)
5 )
```

time_signature	proportion
4	88.765442
3	8.880294
5	1.444528
1	0.890003
0	0.019734

dtype: float64

```
1 temp = (df_song
2     .loc[:,['duration_ms']]
3     .assign(duration_minutes=df_song['duration_ms'].div(1000).div(60))
4     .drop(columns='duration_ms'))
5
```

```
# no. of songs per year
sns.histplot(temp,kde=False,stat='count',bins=temp['duration_minutes'].nun
plt.show()

(
    df_song[['duration_ms']] # Corrected to select as a DataFrame
    .assign(duration_minutes=df_song['duration_ms'].div(1000).div(60))
    .drop(columns='duration_ms')
    .describe()
)
```



	duration_minutes
count	50674.000000
mean	4.185893
std	1.793154
min	0.023983
25%	3.212217
50%	3.915550
75%	4.803046
max	63.606217

```
(
    df_song
    .loc[df_song['duration_ms'].div(1000).div(60) > 60]
)
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms	danceabi
25337	TRDAOJL128F932C383	Dopesmoker	Sleep	1vhvheW4R0KbK6Kr3NFplW	psychedelic, doom_metal	NaN	2003	3816373	

Danceability :
if low -> slowed song
if high -> dancing, party type song

Energy :
if low -> sad, slowed, calm etc
if high -> loud, energetic

Loudness :
low -> low voice song
high -> high voice song

Speechiness :
low -> less lyrics and highly music
high -> highly lyrics

Acousticness :
low -> more electronic or synthetic in nature
high -> featuring instruments like guitar, piano, strings

Instrumentalness :
low -> more electronic or synthetic in nature
high -> featuring instruments like guitar, piano, strings

Liveness :
low -> recorded in studio
high -> recorded infront of live audience

Valence :
low -> sad
high -> happy

Tempo : beats per minute
low -> slow-paced
high -> dance type

```
def numerical_analysis(df,c_name,k=15):
  for i in c_name:
    print('numerical analysis for column ', i)
    print('Stastical summary: ')
    print(df[i].describe())

    fig = plt.figure(figsize=(12,4))
    plt.subplot(1,2,1)
    sns.histplot(df[i],kde=True)
    plt.title(f'Histogram for {i}')
    plt.subplot(1,2,2)
    sns.boxplot(df[i])
    plt.title(f'Boxplot for {i}')
    plt.show()

    print('*'*20)
  print()
  print('pairplot')
  sns.pairplot(df[c_name])
  plt.show()

```

User data

```
from pathlib import Path

users_data_path = Path(path) / 'User Listening History.csv'

df_user = pd.read_csv(users_data_path)
df_user.head(2)
```

	track_id	user_id	playcount
0	TRIRLYL128F42539D1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1
1	TRFUPBA128F934F7E1	b80344d063b5ccb3212f76538f3d9e43d87dca9e	1

```
df_user.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9711301 entries, 0 to 9711300
Data columns (total 3 columns):
 #   Column     Dtype 
---  ------     ----- 
 0   track_id   object
 1   user_id    object
 2   playcount  int64 
dtypes: int64(1), object(2)
memory usage: 222.3+ MB
```

```
df_user.columns
```

```
Index(['track_id', 'user_id', 'playcount'], dtype='object')
```

which user played perticular song how many times that data stored in this dataset

```
df_user.duplicated(subset=['user_id','track_id']).sum()
```

```
np.int64(0)
```

```
df_user.isna().sum()
```

	0
track_id	0
user_id	0
playcount	0

dtype: int64

```
df_user['track_id'].nunique()
```

```
30459
```

```
df_user['user_id'].nunique()
```

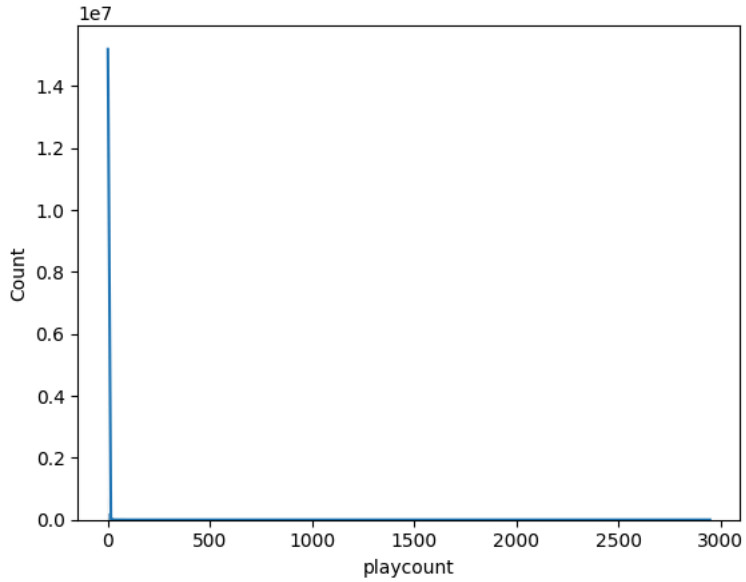
```
962037
```

```
df_user['playcount'].agg(['min', 'max'])
```

	playcount
min	1
max	2948

dtype: int64

```
sns.histplot(df_user['playcount'],kde=True,stat='count',bins=2000)
plt.show()
```




```
# top 10 famous song
top_10_songs = (
    df_user['track_id']
    .value_counts()
    .head(10)
)
top_10_songs
```

track_id	count
TRONYHY128F92C9D11	80656
TRUFTBY128F93450B8	39529
TRXWAZC128F9314B3E	30873
TRCPXID128F92D5D3C	30057
TRGCHLH12903CB7352	29708
TROMKCG128F9320C09	28735
TRPFYYL128F92F7144	28412
TRPGPDK12903CCC651	27276
TRWAQOC12903CB84CA	27222
TRAALAH128E078234A	26689

dtype: int64

```
(
    df_song
    .loc[df_song['track_id'].isin(top_10_songs.index)]
)
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms	da
44	TRAALAH128E078234A	Bitter Sweet Symphony	The Verve	0jLnevC3Vn34qVWrAa4X6x	rock, alternative, indie, pop, alternative_roc...	NaN	1999	358333	
59	TRPFYYL128F92F7144	Float On	Modest Mouse	1Urf1M52P3R6NYdAOJizoW	rock, alternative, indie, alternative_rock, in...	Rock	2004	208466	
323	TRONYHY128F92C9D11	Revelry	Kings of Leon	039Q3UlFQ6kavVIZHpO4mL	rock, alternative, indie, alternative_rock, in...	Rock	2008	201733	
1876	TRXWAZC128F9314B3E	Heartbreak Warfare	John Mayer	0naTARZScsZOtx3nlhlq0Y	rock, alternative, indie, pop, singer_songwrit...	Rock	2010	263280	
2107	TRUFTBY128F93450B8	Alejandro	Lady Gaga	0CXHrBetrvDx4PwBar1ZWj	electronic, pop, female_vocalists, dance	Pop	2010	274800	
2742	TRCPXID128F92D5D3C	Halo	Depeche Mode	0Ti7ZxvgWq74Ls56vYP3Ov	electronic, pop, 80s, british, 90s, new_wave, ...	NaN	1990	270160	
3168	TRWAQOC12903CB84CA	Sexy Bitch	David Guetta	01N6xy2PX9fKVfrA2YOkYd	electronic, dance, house	NaN	2009	193800	
19724	TROMKCG128F9320C09	Uprising	Sabaton	09tHVoXbJNZUotndn8pfJr	metal, heavy_metal, power_metal	NaN	2010	295640	
22071	TRPGPDK12903CCC651	Bring Me To Life	Katherine Jenkins	0rJ8HF2zsxsWMzirj3YFQR	classical, cover, new_age	Rock	2012	226093	
23220	TRGCHLH12903CB7352	Party In The U.S.A.	The Barden Bellas	0bz2Uy1KE7bNGsGQU9pZrU	soundtrack, cover	Pop	2012	63080	

```
top_10_played = (
    df_user.groupby('track_id')['playcount']
    .agg('sum')
    .sort_values(ascending=False)
    .head(10)
)
top_10_played
```

track_id	playcount
TRONYHY128F92C9D11	527893
TRUFTBY128F93450B8	111615
TRZNAHL128F9327D5A	111596
TRCPXID128F92D5D3C	91461
TRPGPDK12903CCC651	91448
TRXWAZC128F9314B3E	87745
TROMKCG128F9320C09	87050
TRPFYYL128F92F7144	85079
TRGCHLH12903CB7352	78443
TRAALAH128E078234A	76893

dtype: int64

```
(
    df_song
    .loc[df_song['track_id'].isin(top_10_played.index)]
)
```

	track_id	name	artist	spotify_id	tags	genre	year	duration_ms	dar
44	TRAALAH128E078234A	Bitter Sweet Symphony	The Verve	0jLnevC3Vn34qVWrAa4X6x	rock, alternative, indie, pop, alternative_roc...	NaN	1999	358333	
59	TRPFYYL128F92F7144	Float On	Modest Mouse	1Urf1M52P3R6NYdAOJizoW	rock, alternative, indie, alternative_rock, in...	Rock	2004	208466	
323	TRONYHY128F92C9D11	Revelry	Kings of Leon	039Q3UlFQ6kavVIZHpO4mL	rock, alternative, indie, alternative_rock, in...	Rock	2008	201733	
1876	TRXWAZC128F9314B3E	Heartbreak Warfare	John Mayer	0naTARZScsZOtx3nlhlq0Y	rock, alternative, indie, pop, singer_songwrit...	Rock	2010	263280	
2107	TRUFTBY128F93450B8	Alejandro	Lady Gaga	0CXHrBetrvDx4PwBar1ZWj	electronic, pop, female_vocalists, dance	Pop	2010	274800	
2742	TRCPXID128F92D5D3C	Halo	Depeche Mode	0Ti7ZxvgWq74Ls56vYP3Ov	electronic, pop, 80s, british, 90s, new_wave, ...	NaN	1990	270160	
19724	TROMKCG128F9320C09	Uprising	Sabaton	09tHVoXbJNZUotndn8pfJr	metal, heavy_metal, power_metal	NaN	2010	295640	
22071	TRPGPDK12903CCC651	Bring Me To Life	Katherine Jenkins	0rJ8HF2zsxsWMzirj3YFQR	classical, cover, new_age	Rock	2012	226093	
23220	TRGCHLH12903CB7352	Party In The U.S.A.	The Barden Bellas	0bz2Uy1KE7bNGsGQU9pZrU	soundtrack, cover	Pop	2012	63080	
25264	TRZNAHL128F9327D5A	Gears	Miss May I	3YC9z1sMjAxn5noHpBLBXd	metalcore, post_hardcore, screamo	NaN	2010	241466	

```
pd.concat([top_10_songs,top_10_played,],axis=1)
```

track_id	count	playcount
TRONYHY128F92C9D11	80656.0	527893.0
TRUFTBY128F93450B8	39529.0	111615.0
TRXWAZC128F9314B3E	30873.0	87745.0
TRCPXID128F92D5D3C	30057.0	91461.0
TRGCHLH12903CB7352	29708.0	78443.0
TROMKCG128F9320C09	28735.0	87050.0
TRPFYYL128F92F7144	28412.0	85079.0
TRPGPDK12903CCC651	27276.0	91448.0
TRWAQOC12903CB84CA	27222.0	NaN
TRAALAH128E078234A	26689.0	76893.0
TRZNAHL128F9327D5A	NaN	111596.0

```
# user who played diff.(unique) songs

most_divers_users = (
    df_user
    .groupby('user_id')['track_id']
    .agg('count')
    .sort_values(ascending=False)
    .head(10)
)
most_divers_users
```

user_id	track_id
ec6dfcf19485cb011e0b22637075037aae34cf26	784
4e11f45d732f4861772b2906f81a7d384552ad12	384
726da71c2c2ea119119a7957517fccd028d1be76	376
113255a012b2affeab62607563d03fbdf31b08e7	367
7adec7f006cb09482d36609d205293d8b61f030e	366
fef771ab021c200187a419f5e55311390f850a50	363
8cb51abc6bf8ea29341cb070fe1e1af5e4c3ffcc	362
b4c94d72b15d3c311c10045a58b31f95d9d12785	357
96f7b4f800cafef33eae71a6bc44f7139f63cd7a	356
6d625c6557df84b60d90426c0116138b617b9449	350

dtype: int64

```
most_active_user = (
    df_user.groupby('user_id')['playcount']
    .agg('sum')
    .sort_values(ascending=False)
    .head(10)
)
most_active_user
```

user_id	playcount
1854daf178674bbac9a8ed3d481f95b76676b414	2953
944cdf52364f45b0edd1c972b5a73d3a86b09c6a	2046
6a8a142084a4818c0dcac48bdfb3c39deacf5168	1942
93158e3983ffc8945e25c793d93d6b67d46cae9d	1845
f0b5c784daaa3c7d50acbb3723c7dd649db2b231	1841
6a58f480d522814c087fd3f8c77b3f32bb161f9d	1783
49127655d27dabab3f469bcdd996330fe4f3e210	1743
3fa44653315697f42410a30cb766a4eb102080bb	1639
839223f11c98e0c8017e8ecd6fc7b8706658c966	1608
af3ee32357049dd96231238bd1b019e8142ee6aa	1579

dtype: int64

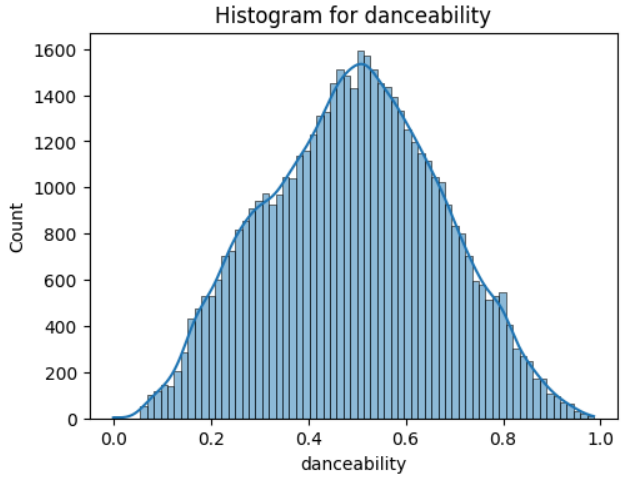
```
pd.concat([most_divers_users,most_active_user],axis=1)
```

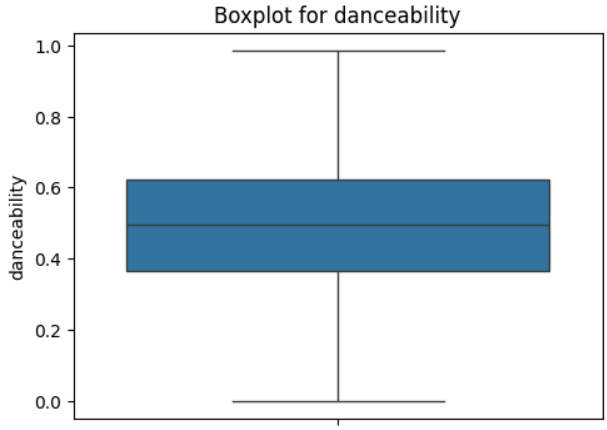
user_id	track_id	playcount
ec6dfcf19485cb011e0b22637075037aae34cf26	784.0	NaN
4e11f45d732f4861772b2906f81a7d384552ad12	384.0	NaN
726da71c2c2ea119119a7957517fccd028d1be76	376.0	NaN
113255a012b2affeab62607563d03fbdf31b08e7	367.0	NaN
7adec7f006cb09482d36609d205293d8b61f030e	366.0	NaN
fef771ab021c200187a419f5e55311390f850a50	363.0	NaN
8cb51abc6bf8ea29341cb070fe1e1af5e4c3ffcc	362.0	NaN
b4c94d72b15d3c311c10045a58b31f95d9d12785	357.0	NaN
96f7b4f800cafef33eae71a6bc44f7139f63cd7a	356.0	NaN
6d625c6557df84b60d90426c0116138b617b9449	350.0	NaN
1854daf178674bbac9a8ed3d481f95b76676b414	NaN	2953.0
944cdf52364f45b0edd1c972b5a73d3a86b09c6a	NaN	2046.0
6a8a142084a4818c0dcac48bdfb3c39deacf5168	NaN	1942.0
93158e3983ffc8945e25c793d93d6b67d46cae9d	NaN	1845.0
f0b5c784daaa3c7d50acbb3723c7dd649db2b231	NaN	1841.0
6a58f480d522814c087fd3f8c77b3f32bb161f9d	NaN	1783.0
49127655d27dabab3f469bcdd996330fe4f3e210	NaN	1743.0
3fa44653315697f42410a30cb766a4eb102080bb	NaN	1639.0
839223f11c98e0c8017e8ecd6fc7b8706658c966	NaN	1608.0
af3ee32357049dd96231238bd1b019e8142ee6aa	NaN	1579.0

```
Start coding or generate with AI.
```

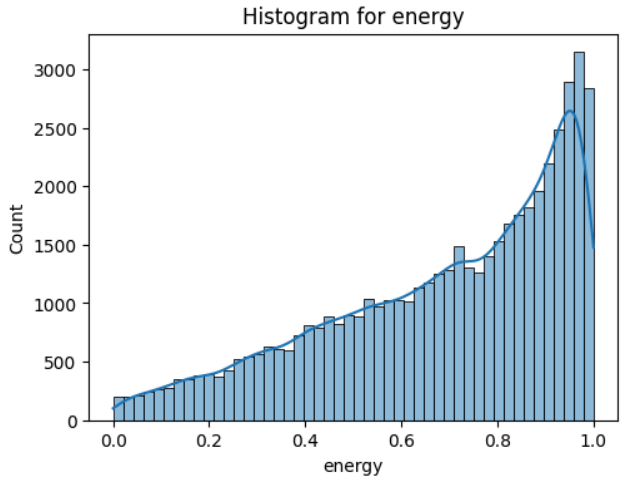
```
float_col = df_song.select_dtypes(include='float').columns
numerical_analysis(df_song,float_col)
```

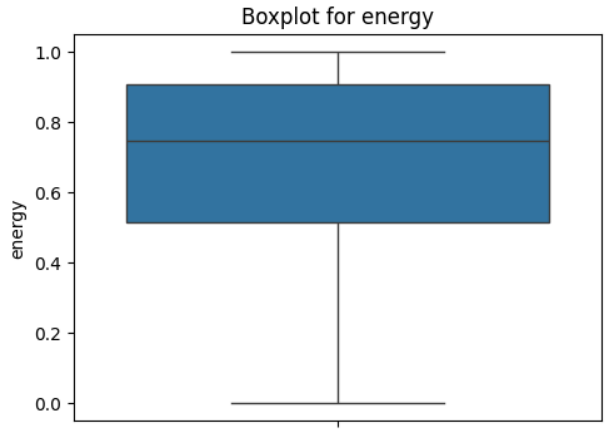
```
numerical analysis for column  danceability
Stastical summary:
count    50674.000000
mean         0.493522
std          0.178833
min          0.000000
25%          0.364000
50%          0.497000
75%          0.621000
max          0.986000
Name: danceability, dtype: float64
```



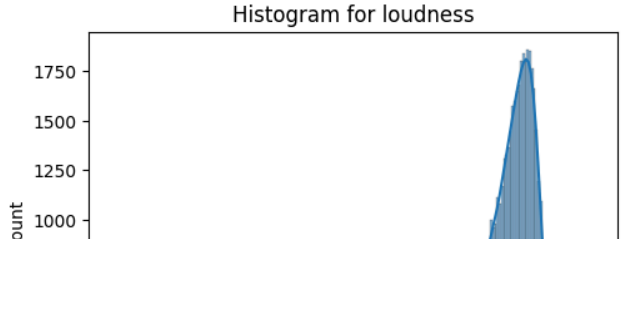


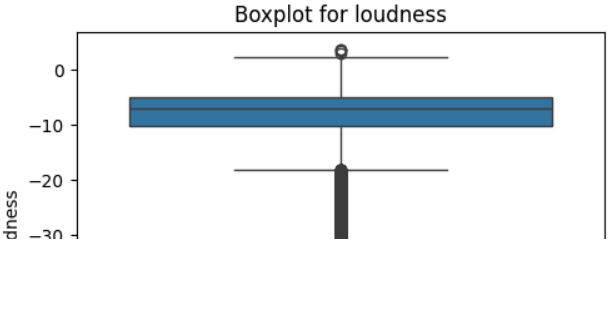

```
********************
numerical analysis for column  energy
Stastical summary:
count    50674.000000
mean         0.686507
std          0.251803
min          0.000000
25%          0.514000
50%          0.744000
75%          0.905000
max          1.000000
Name: energy, dtype: float64
```

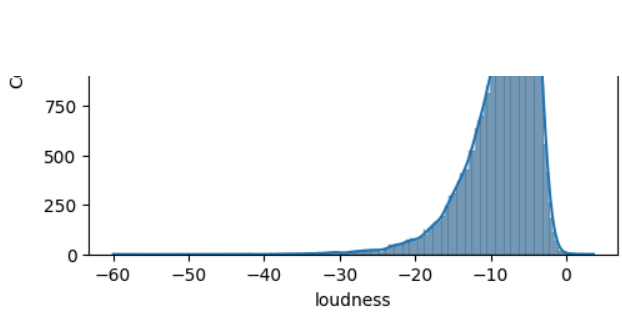


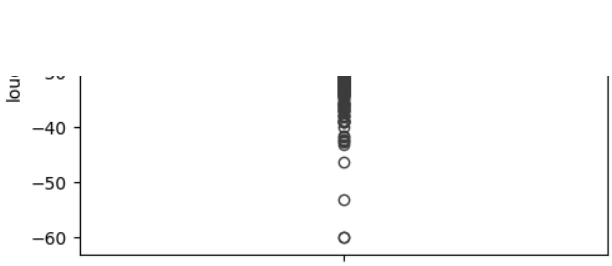



```
********************
numerical analysis for column  loudness
Stastical summary:
count    50674.000000
mean        -8.291007
std          4.548359
min        -60.000000
25%        -10.375000
50%         -7.199500
75%         -5.089000
max          3.642000
Name: loudness, dtype: float64
```

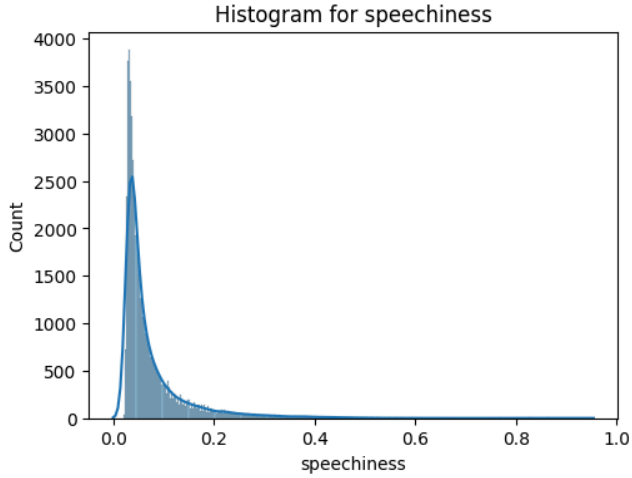


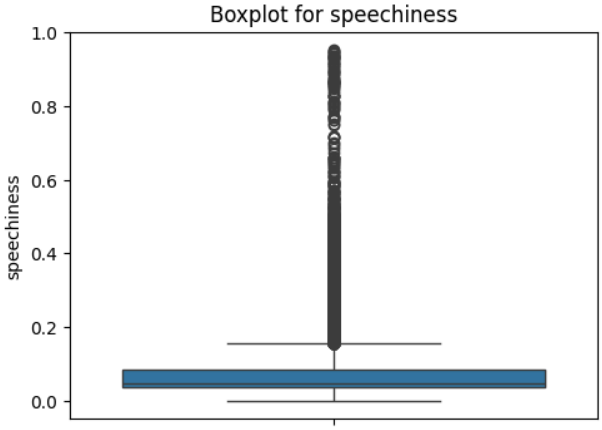




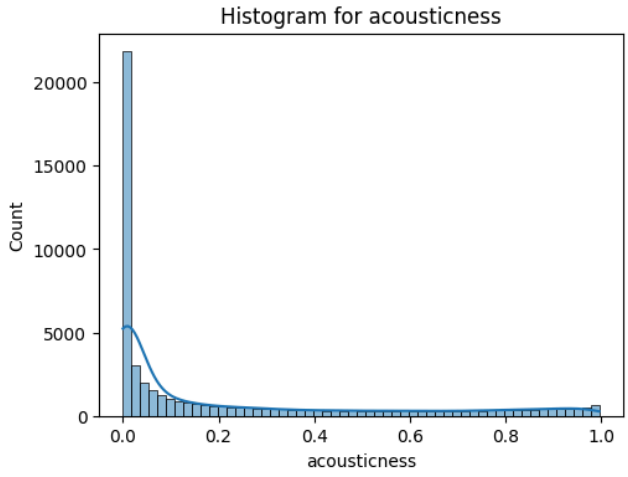


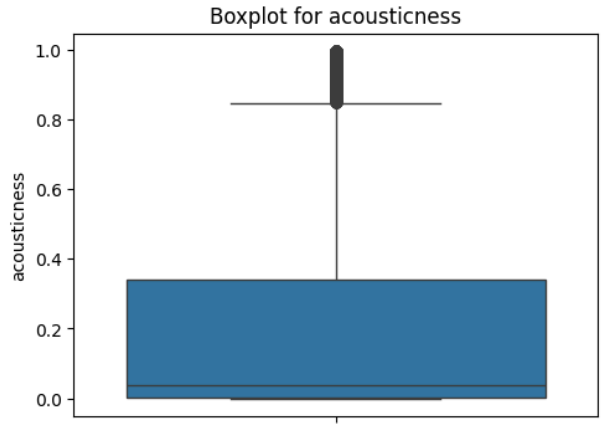

```
********************
numerical analysis for column  speechiness
Stastical summary:
count    50674.000000
mean         0.076026
std          0.076012
min          0.000000
25%          0.035200
50%          0.048200
75%          0.083500
max          0.954000
Name: speechiness, dtype: float64
```



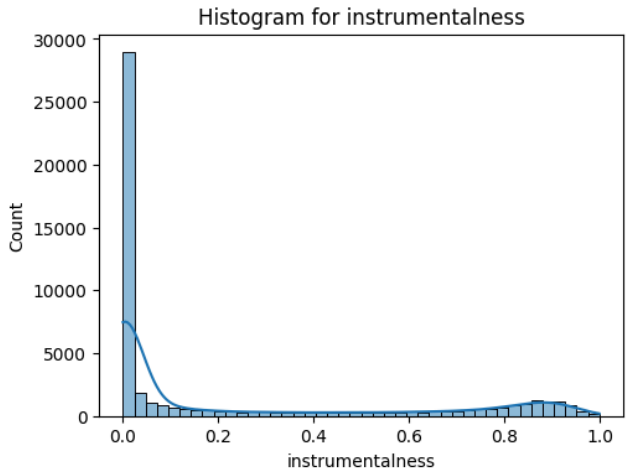


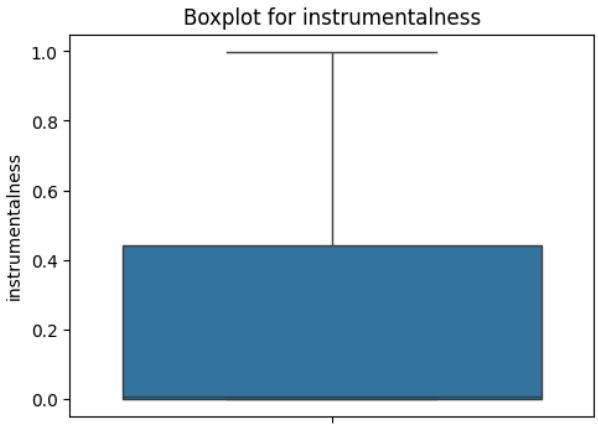

```
********************
numerical analysis for column  acousticness
Stastical summary:
count    50674.000000
mean         0.213798
std          0.302839
min          0.000000
25%          0.001400
50%          0.039900
75%          0.340000
max          0.996000
Name: acousticness, dtype: float64
```



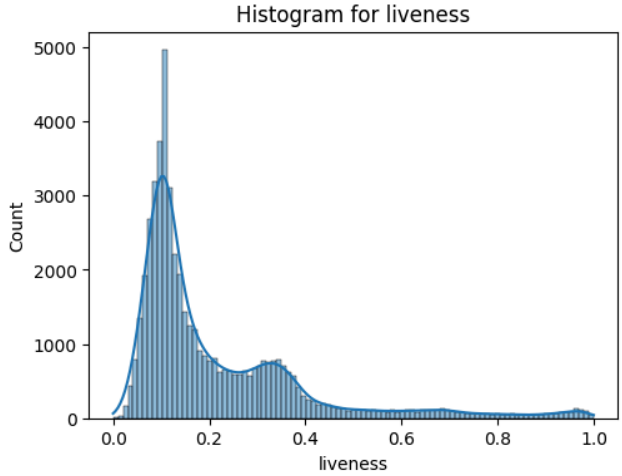


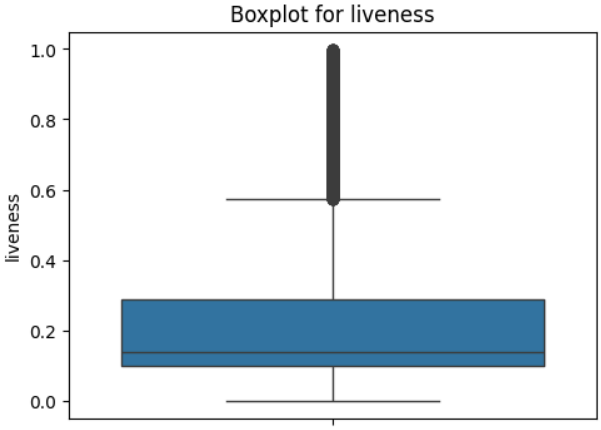

```
********************
numerical analysis for column  instrumentalness
Stastical summary:
count    50674.000000
mean         0.225299
std          0.337067
min          0.000000
25%          0.000018
50%          0.005630
75%          0.441000
max          0.999000
Name: instrumentalness, dtype: float64
```



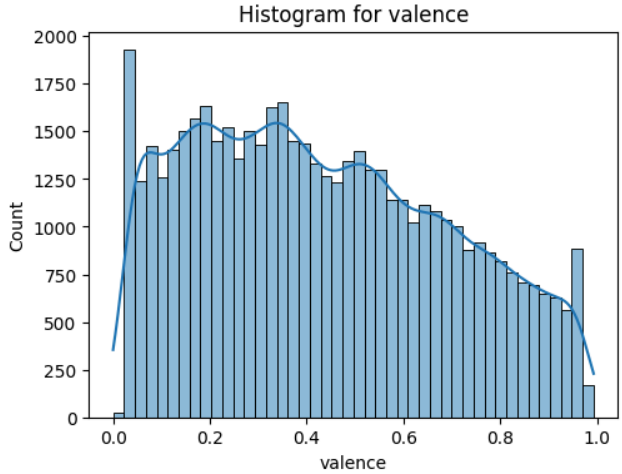


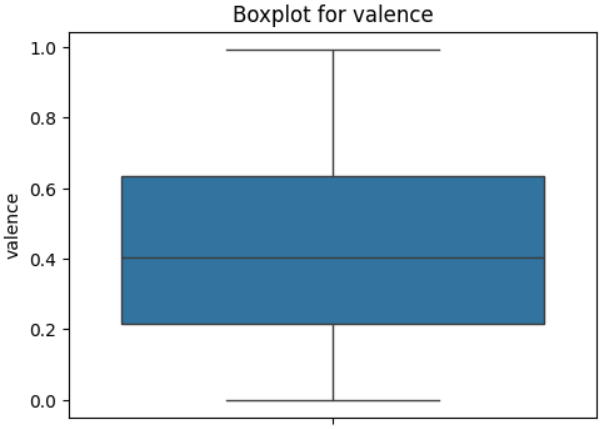
****
numerical analysis for column liveness
Stastical summary:
count   50674.000000
mean   0.215439
std   0.184708
min   0.000000
25%   0.098400
50%   0.138000
75%   0.289000
max   0.999000
Name: liveness, dtype: float64





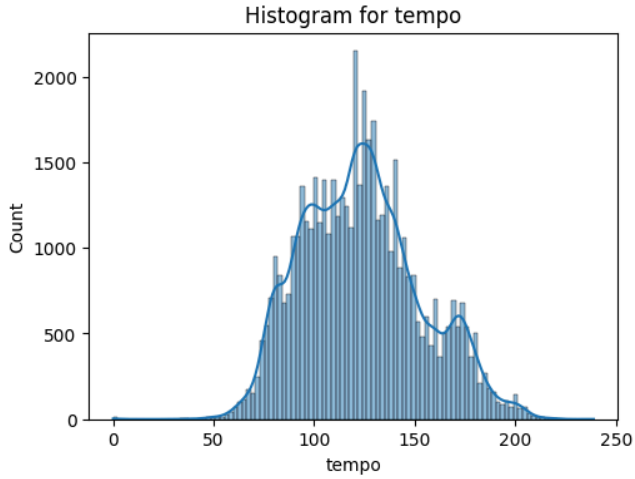
****
numerical analysis for column valence
Stastical summary:
count   50674.000000
mean   0.433113
std   0.258767
min   0.000000
25%   0.214000
50%   0.405000
75%   0.634000
max   0.993000
Name: valence, dtype: float64

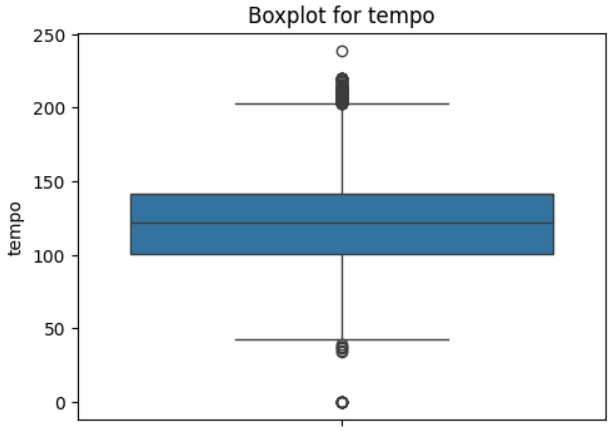




****

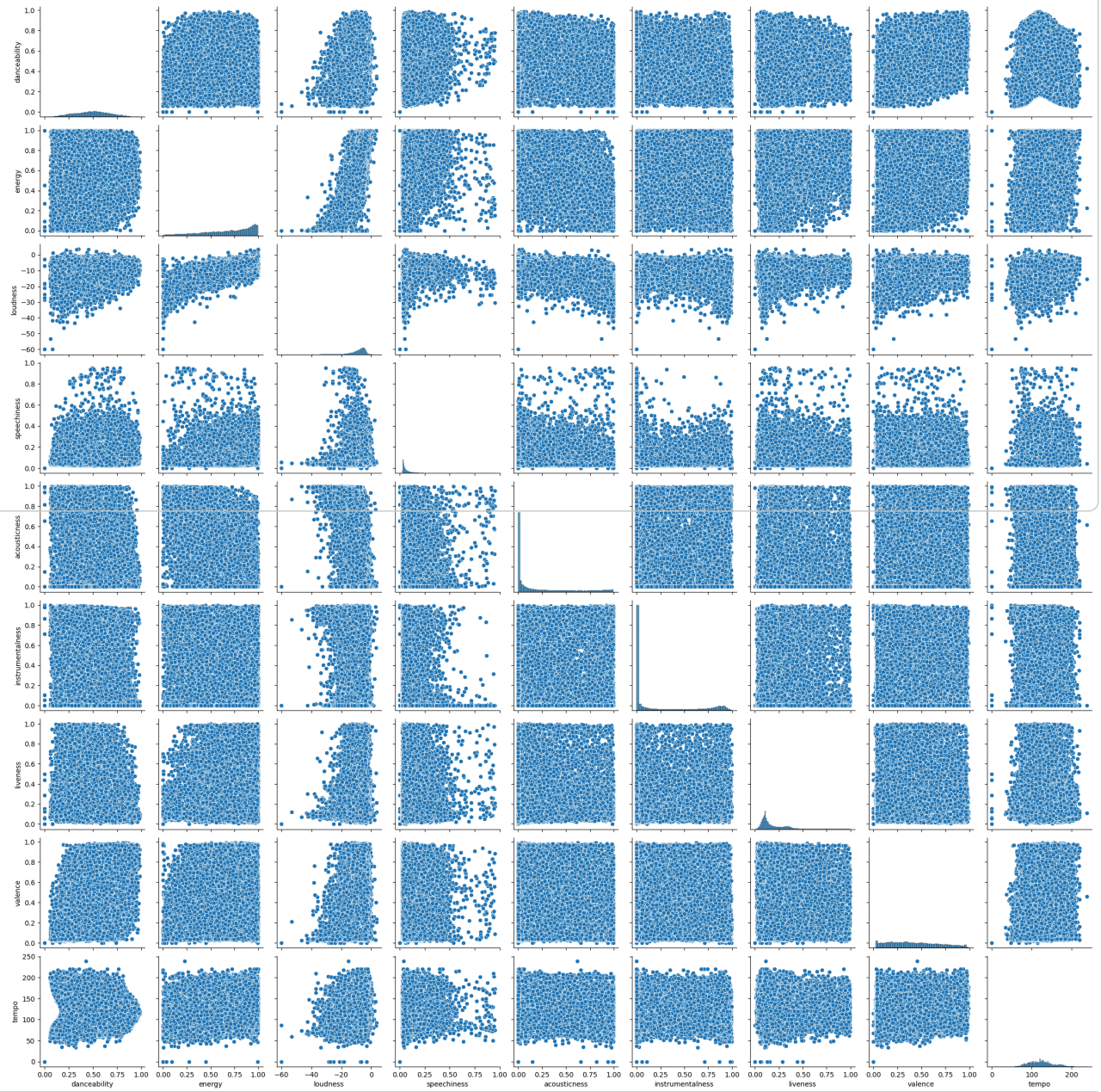
```
numerical analysis for column  tempo
Stastical summary:
count    50674.000000
mean       123.508794
std         29.622349
min          0.000000
25%        100.682500
50%        121.989000
75%        141.642250
max        238.895000
Name: tempo, dtype: float64
```





****

pairplot



Data Cleaning & similarity calculating

```
from pathlib import Path

songs_data_path = Path(path) / 'Music Info.csv'

df_song = pd.read_csv(songs_data_path)
df_song.head(2)
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre	year
0	TRIOREW128F424EAF0	Mr. Brightside	The Killers	https://p.scdn.co/mp3-preview/4d26180e6961fd46...	09ZQ5TmUG8TSL56n0knqrj	rock, alternative, indie, alternative_rock, in...	NaN	2004
1	TRRIVDJ128F429B0E8	Wonderwall	Oasis	https://p.scdn.co/mp3-preview/d012e536916c927b...	06UfBBDISthj1ZJAtX4xjj	rock, alternative, indie, pop, alternative_roc...	NaN	2006

2 rows × 21 columns

```
df_song.isna().sum()
```

	0
track_id	0
name	0
artist	0
spotify_preview_url	0
spotify_id	0
tags	1127
genre	28335
year	0
duration_ms	0
danceability	0
energy	0
key	0
loudness	0
mode	0
speechiness	0
acousticness	0
instrumentalness	0
liveness	0
valence	0
tempo	0
time_signature	0

dtype: int64

```
df_song.duplicated(subset='spotify_id').sum()
```

```
np.int64(9)
```

```
df_song.reset_index(drop=True,inplace=True)
```

```
# droping some column which have unique values
col = ['spotify_id','track_id','genre','name','spotify_preview_url']
df_cleaned = df_song.drop(columns=col, errors='ignore', inplace=False) # Cl
```

```
# replacing null values in tags column with no tag
df_cleaned['tags'] = df_cleaned['tags'].fillna('no_tag')
```

```
# converting some columns in lower case
df_cleaned['artist'] = df_cleaned['artist'].str.lower()
```

```
df_song.head(2)
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre	year
0	TRIOREW128F424EAF0	Mr. Brightside	The Killers	https://p.scdn.co/mp3-preview/4d26180e6961fd46...	09ZQ5TmUG8TSL56n0knqrj	rock, alternative, indie, alternative_rock, in...	NaN	2004
1	TRRIVDJ128F429B0E8	Wonderwall	Oasis	https://p.scdn.co/mp3-preview/d012e536916c927b...	06UfBBDISthj1ZJAtX4xjj	rock, alternative, indie, pop, alternative_roc...	NaN	2006

2 rows × 21 columns

```
# tags used more that 1000 songs
(
    df_cleaned['tags']
    .str.lower()
    .str.split(',')
    .explode()
    .str.strip()
    .value_counts()
    .loc[lambda ser: ser >= 1000]
)
```

tags	count
rock	10684
indie	7287
electronic	6594
alternative	6274
pop	4651
...	...
ska	1089
gothic_metal	1072
grindcore	1040
french	1018
nu_metal	1006

86 rows × 1 columns

dtype: int64

```
!pip install category_encoders -q
from category_encoders.count import CountEncoder
from sklearn.preprocessing import OneHotEncoder,MinMaxScaler,StandardScale
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.compose import ColumnTransformer
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.9/85.9 kB 3.0 MB/s eta 0:00:00
```

```
df_cleaned.head(2)
```

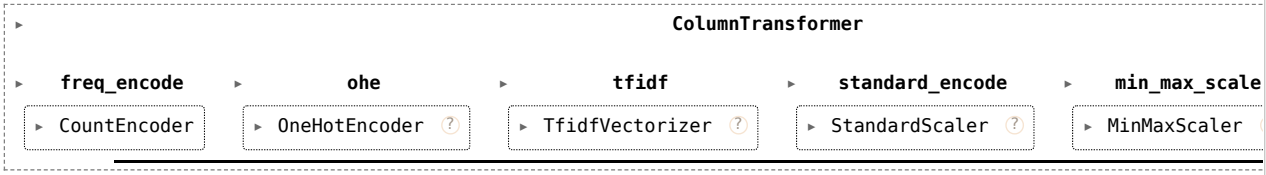
	artist	tags	year	duration_ms	danceability	energy	key	loudness	mode	speechiness	acousticness	in
0	the killers	rock, alternative, indie, alternative_rock,	2004	222200	0.355	0.918	1	-4.360	1	0.0746	0.001190	

Next steps: Generate code with df_cleaned | New interactive sheet

```
scaler1 = MinMaxScaler()
scaler2 = StandardScaler()
ohe = OneHotEncoder(handle_unknown='ignore')
tfidf = TfidfVectorizer(stop_words='english',max_features=85)

freq_encode_col = ['year'] # range : 0 to 1
ohe_cols = ['artist','time_signature','key']
tfidf_col = 'tags'
standard_encode_col = ['duration_ms','loudness','tempo']
min_max_scale_col = ['danceability','energy','speechiness','acousticness',

len(freq_encode_col+ohe_cols+standard_encode_col+min_max_scale_col)
```

```
14
```

```
# transform the data

transformer = ColumnTransformer([
    ('freq_encode',CountEncoder(normalize=True,return_df=True),freq_encode
    ('ohe',ohe,ohe_cols),
    ('tfidf',tfidf, tfidf_col),
    ('standard_encode',scaler2,standard_encode_col),
    ('min_max_scale',scaler1,min_max_scale_col)
],remainder='passthrough',n_jobs=-1,force_int_remainder_cols=False)
transformer
```




```
df_cleaned.shape
```

```
(50683, 16)
```

```
df_transformed = transformer.fit_transform(df_cleaned)
df_transformed.shape
```

```
(50683, 8431)
```

```
df_song.loc[df_song['artist']=='Shakira'].head(2)
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre
1025	TRLWDVU128F932B093	Whenever, Wherever	Shakira	https://p.scdn.co/mp3-preview/09ddeb4ae33ee6e8...	07PHBDuUmOeZ7jeKSbAbKi	rock, pop, female_vocalists, singer_songwriter...	NaN
2068	TRILOWN128F426080F	Underneath Your Clothes	Shakira	https://p.scdn.co/mp3-preview/6c5a56058ce04371...	07qRl4PT2lA6O3KN40McLz	rock, pop, female_vocalists, singer_songwriter...	Pop

2 rows × 21 columns

```
df_song.loc[df_song['name']=='Whenever, Wherever']
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre
1025	TRLWDVU128F932B093	Whenever, Wherever	Shakira	https://p.scdn.co/mp3-preview/09ddeb4ae33ee6e8...	07PHBDuUmOeZ7jeKSbAbKi	rock, pop, female_vocalists, singer_songwriter...	NaN

1 rows × 21 columns

```
df_song[df_song['name']=='Whenever, Wherever']
```

	track_id	name	artist	spotify_preview_url	spotify_id	tags	genre
1025	TRLWDVU128F932B093	Whenever, Wherever	Shakira	https://p.scdn.co/mp3-preview/09ddeb4ae33ee6e8...	07PHBDuUmOeZ7jeKSbAbKi	rock, pop, female_vocalists, singer_songwriter...	NaN

1 rows × 21 columns

```
# song_index = df_song[df_song['name']=='Whenever, Wherever'].index
# song_input = df_cleaned.loc[song_index]
# song_input
song_input = df_cleaned[df_song['name']=='Whenever, Wherever']
song_input
```

	artist	tags	year	duration_ms	danceability	energy	key	loudness	mode	speechiness	acousticnes
1025	shakira	rock, pop, female vocalists,	2012	196826	0.787	0.828	1	-4.967	0	0.0474	0.29

```
input_vector = transformer.transform(song_input)
input_vector
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 20 stored elements and shape (1, 8431)>
```

```
similarity_score = cosine_similarity(df_transformed,input_vector)
similarity_score.shape
```

```
(50683, 1)
```

```
top_10 = np.argsort(similarity_score.ravel())[-11:][::-1]
top_10
```

```
array([ 1025, 12306,  6047,  6130, 17243,  6134,  7173,  6122,  6527,
       38389,  6288])
```

```
top_10_song = df_song.iloc[top_10]
top_10_song
```